中山大学计算机学院本实验报告

（2025 学年春季学期）

课程名称:	并行程序设计与算法
实验题目:	Lab5 基于 OpenMP 的并行矩阵乘法
批改人:	
专业（方向）:	
学号:	23336128
姓名:	梁力航
Email:	
完成日期:	2026 年 4 月 30 日

本报告围绕 Lab5 的两条主线展开：其一是使用 OpenMP 完成通用矩阵乘法，比较默认调度、`schedule(static,1)` 和 `schedule(dynamic,1)` 三种调度方式；其二是基于 Pthreads 构造 `parallel_for` 动态库，在 Linux 环境下生成 `libparallel_for.so`，并用其并行化矩阵乘法。实验目标不仅是得到可运行程序，更重要的是分析不同并行运行时、不同线程数和不同任务分配策略对性能曲线的影响。

1 实验目的

本实验的目标包括：

1. 使用 OpenMP 实现通用矩阵乘法，并比较三种调度策略；
2. 使用 Pthreads 实现基础版 `parallel_for` 并生成 Linux 动态链接库；
3. 将 `parallel_for` 应用于矩阵乘法，验证其正确性和可用性；
4. 对 4 个版本在不同线程数、不同矩阵规模下进行统一 benchmark；
5. 形成代码、数据、图表与实验报告的完整闭环。

2 实验平台与环境

表 1: 实验环境信息

项目	配置
宿主机	macOS / Darwin 25.3.0 / arm64
canonical 运行环境	Docker Linux (arm64)
C++ 编译器	`g++`，`-O3 -std=c++17 -fopenmp -pthread`
并行模型	OpenMP / Pthreads
Python 工作流	`uv run python ...`
默认随机种子	`20250401`
输出契约	稳定 `key=value` 字段

这里有两个环境事实需要提前说明：

- 宿主机是 macOS，但题目要求 `parallel_for` 生成 Linux 下的 `.so` 文件，因此本实验把 Docker Linux 作为唯一的正式运行口径。
- 正式 benchmark 覆盖 4 个版本、8 个线程配置和 3 个矩阵规模，共 96 条记录，且所有记录状态均为 `ok`，说明这套闭环在工程上是稳定可复现的。

3 程序逻辑流程

3.1 统一运行入口与 OpenMP 执行流程

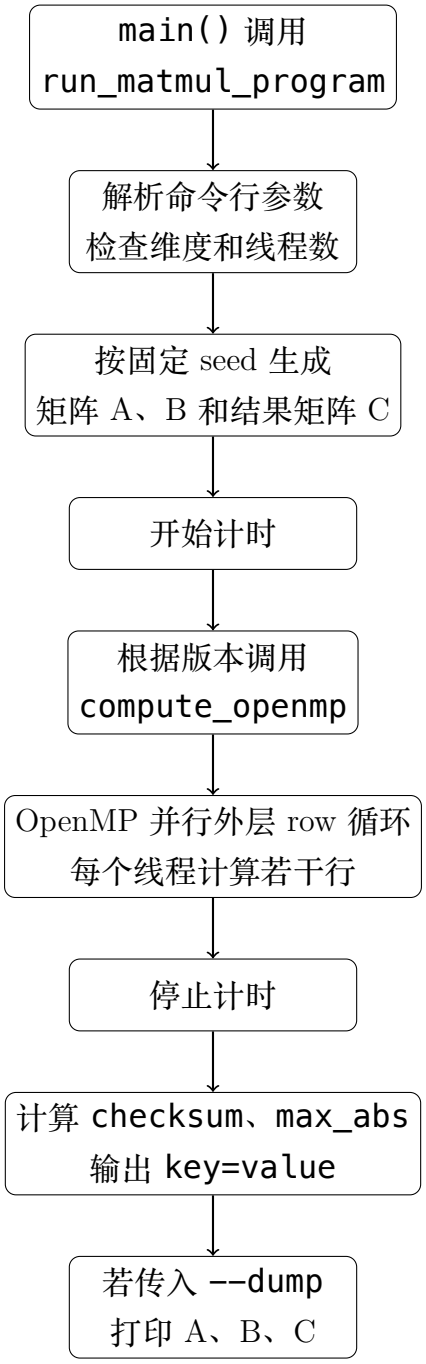


图 1: 统一运行入口与 OpenMP 版本代码逻辑流程

3.2 测试范围

本实验采用如下 benchmark 维度:

- 版本：
  `openmp_default`
  `openmp_static1`
  `openmp_dynamic1`
  `parallel_for_row_block`
- 线程数：$1, 2, 3, 4, 5, 6, 7, 8$
- 矩阵规模：$512, 1024, 2048$
- 每组重复次数：3
- 固定随机种子：20250401

4 核心实现与关键代码

4.1 统一矩阵计算协议

4 个版本共享同一套输入输出和数据生成协议：输入固定为 `m n k seed threads [--dump]`，输出统一为 `experiment`、`backend`、`version`、`time_sec`、`checksum`、`max_abs` 等字段。这样 benchmark 不需要针对每个版本写不同解析逻辑，也保证了跨版本比较的可重复性。

4.2 OpenMP 版本：并行外层行循环

OpenMP 三个版本共用同一份矩阵乘法核心代码，只通过编译期宏切换调度方式。下面给出核心并行循环片段：

```
#if OPENMP_SCHEDULE_KIND == 0
#pragma omp parallel for num_threads(config.threads)
#elif OPENMP_SCHEDULE_KIND == 1
#pragma omp parallel for schedule(static, 1) num_threads(config.
   threads)
#else
#pragma omp parallel for schedule(dynamic, 1) num_threads(config.
   threads)
#endif
for (int row = 0; row < config.m; ++row) {
   for (int col = 0; col < config.k; ++col) {
      double sum = 0.0;
      for (int p = 0; p < config.n; ++p) {
         sum += a[row * config.n + p] * b[p * config.k + col];
      }
```

```
        c[row * config.k + col] = sum;
    }
}
```

之所以只并行最外层行循环，是因为每一行结果相互独立，不会产生写冲突，也不需要额外同步。对本实验的规则矩阵乘法任务来说，这已经足够满足“可以并行执行”的条件。

4.3 parallel_for 动态库：任务分解与执行

parallel_for 的职责不是关心矩阵乘法细节，而是把一段可并行的 for 循环按照线程数拆成若干连续块，然后让多个 Pthreads 线程去执行回调函数。核心逻辑如下：

```
const int total_iters = (end - start + inc - 1) / inc;
const int worker_count = total_iters < num_threads ? total_iters :
    num_threads;
const int base = total_iters / worker_count;
const int extra = total_iters % worker_count;

for (int tid = 0; tid < worker_count; ++tid) {
    const int count = base + (tid < extra ? 1 : 0);
    contexts[tid] = ParallelForWorkerContext{
        start, inc, iter_begin, iter_begin + count, functor, arg,
    };
    iter_begin += count;
    pthread_create(&threads[tid], nullptr, parallel_for_worker, &
        contexts[tid]);
}
```

这个实现只支持连续块划分，不额外提供动态调度或工作窃取机制，因此它的定位是“手工构造并行 for 机制”的教学版实现，而不是成熟的高性能运行时。

4.4 parallel_for 库内部调度流程

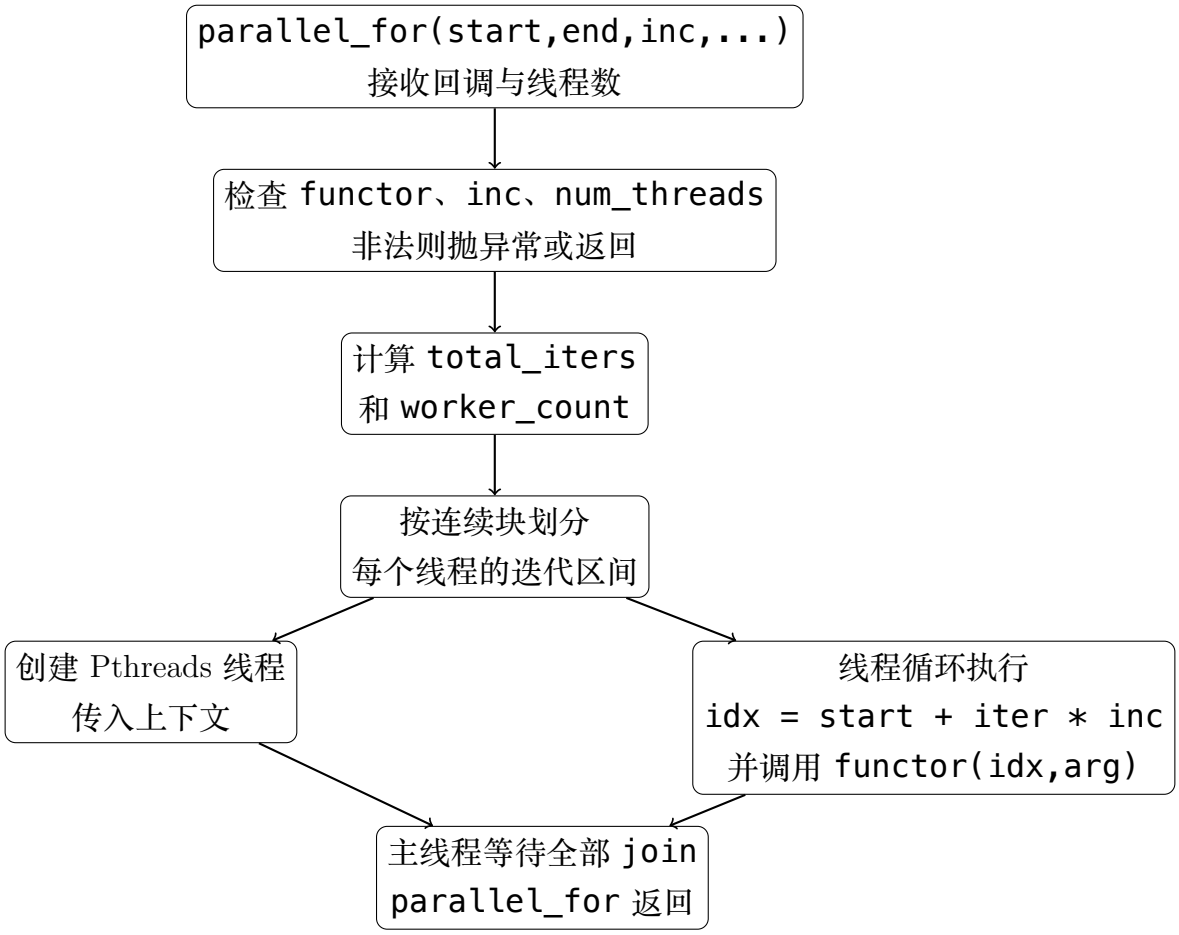


图 2: parallel_for 库内部任务划分与执行流程

4.5 矩阵乘法回调代码片段

为了把矩阵乘法接入 parallel_for，本实验把“按行计算一整行 C”封装成回调函数：

```
static void* compute_row(int row, void* raw_args) {
   auto* args = static_cast<MatmulFunctorArgs*>(raw_args);
   for (int col = 0; col < args->k; ++col) {
      double sum = 0.0;
      for (int p = 0; p < args->n; ++p) {
         sum += args->a[row * args->n + p] * args->b[p * args->k +
            col];
      }
      args->c[row * args->k + col] = sum;
   }
   return nullptr;
}
```

这段代码和 OpenMP 版本的核心计算完全一致，唯一差异在于“谁来调度行循环”。因此 OpenMP 与 `parallel_for` 之间的性能差异，主要来自调度与线程管理机制，而不是矩阵乘法公式本身。

4.6 `parallel_for_matmul` 的行回调映射逻辑

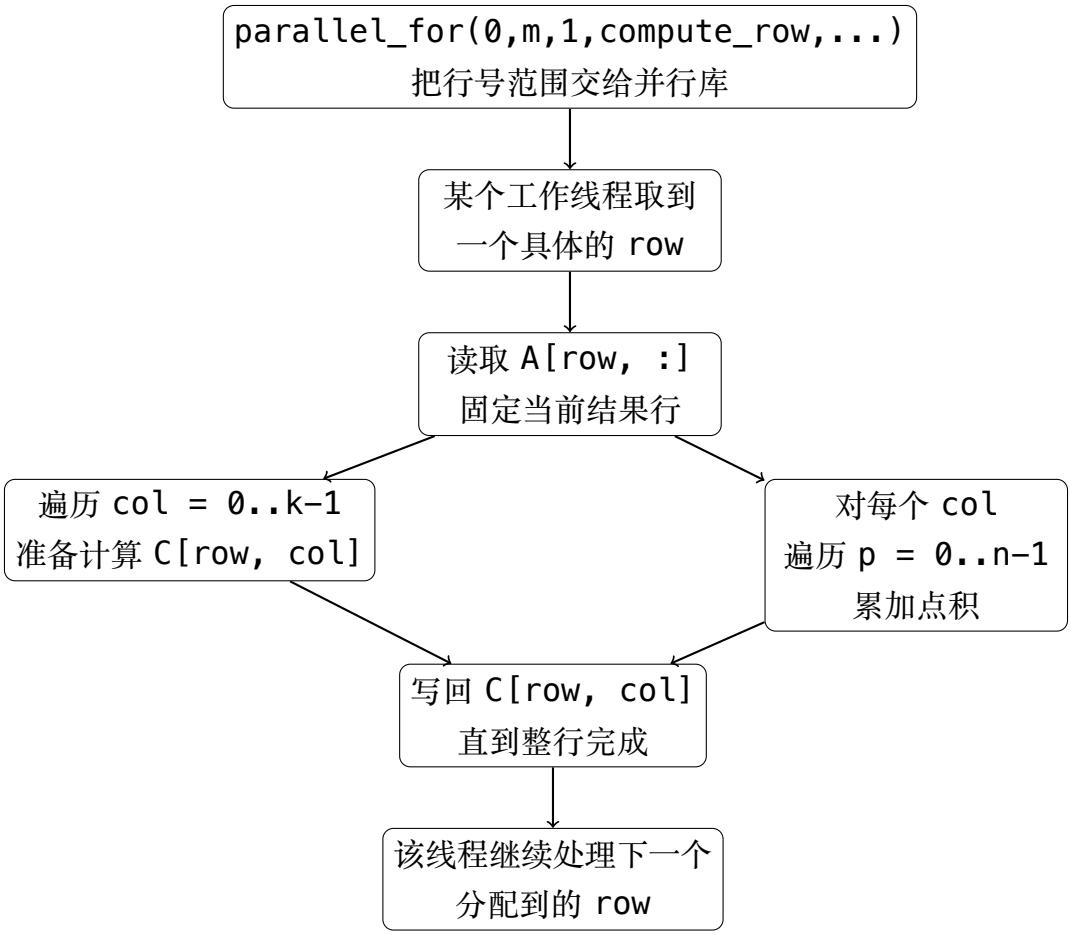


图 3: `parallel_for_matmul` 中“行号回调”到结果矩阵整行计算的映射

这张图强调的是 `parallel_for_matmul.cpp` 中最关键的实现选择：并行库只负责发放行号，真正的矩阵乘法语义保留在 `compute_row` 回调里。也就是说，线程拿到的不是一段原始指针运算，而是“把第 row 行完整算完”这个更清晰的任务单元，因此既避免了写冲突，也让代码结构和串行版本保持一致。

5 关键结果摘要

表 2: 2048 规模下各版本最优结果摘要

版本	最优线程数	单线程时间 (s)	最优时间 (s)	最优加速比	观察
default	6	24.3429	12.5710	1.936	整体最优，表现最稳定
static(1)	8	27.8520	13.4750	2.067	与默认调度接近，后期扩展较平稳
dynamic(1)	4	29.8222	14.2550	2.092	小线程数下收益明显，但高线程数波动较大
parallel_for	8	28.3465	16.3184	1.737	正确性成立，但扩展效率弱于 OpenMP

从表中可以直接看出，本实验里最快的实现是 `openmp_default`。值得注意的是，虽然 `openmp_dynamic1` 的最佳加速比略高于 `openmp_default`，但它的单线程基线本来更慢，因此最终绝对运行时间仍然落后。对实验报告而言，绝对运行时间和扩展趋势比单独的 speedup 数字更有解释力。

6 实验结果与图表分析

6.1 OpenMP 调度策略比较

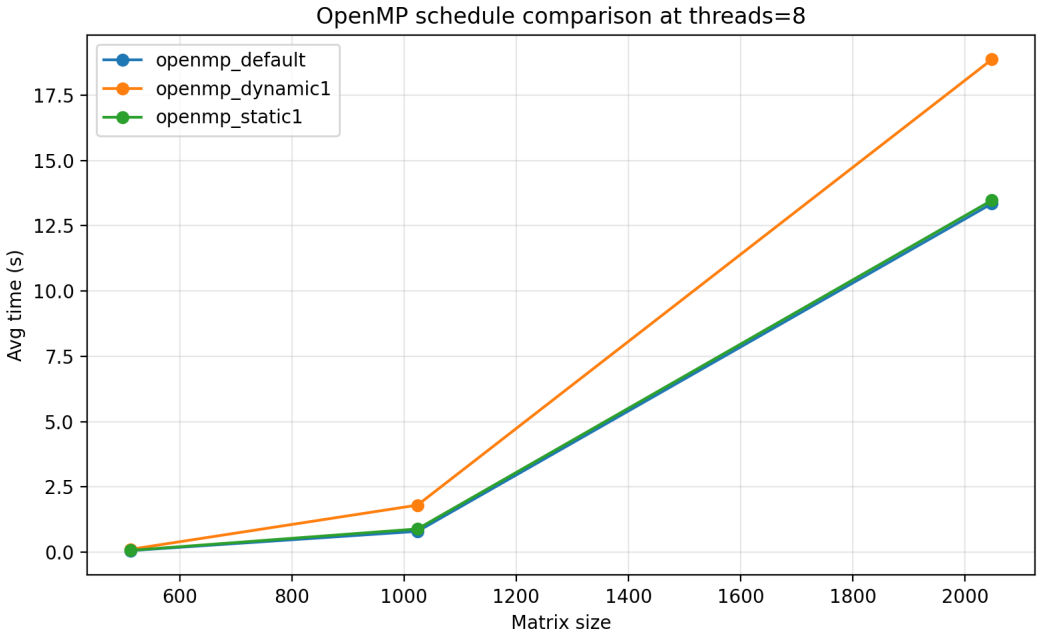
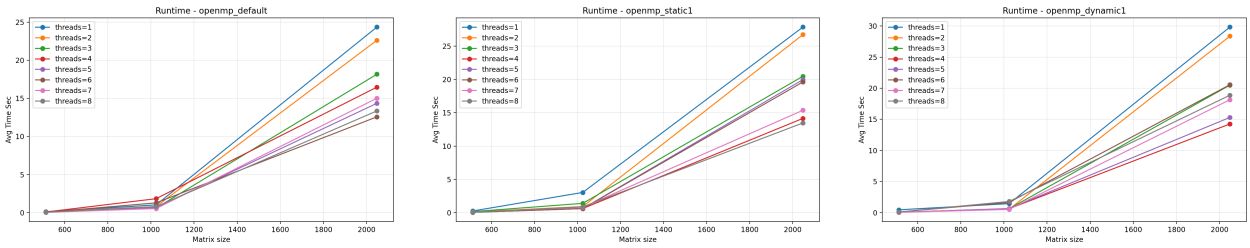
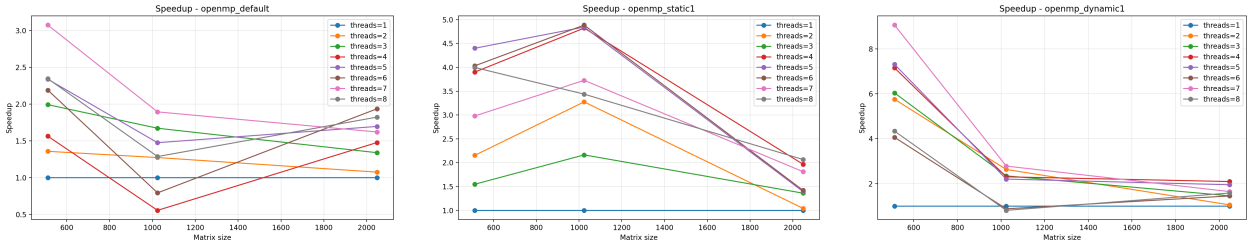


图 4: OpenMP 三种调度策略对比



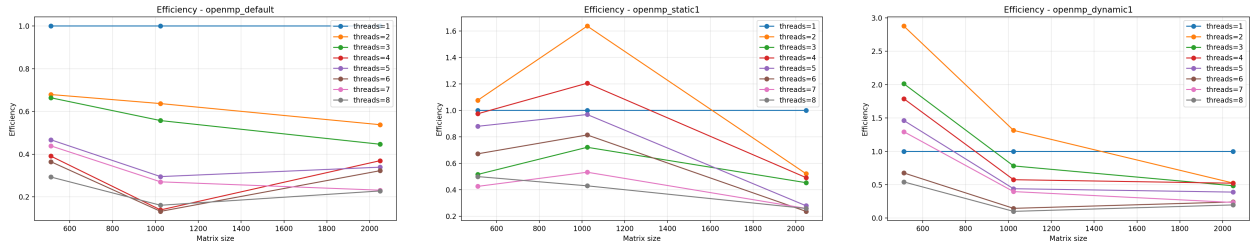
(a) default (b) static(1) (c) dynamic(1)

图 5: OpenMP 三种调度策略运行时间曲线



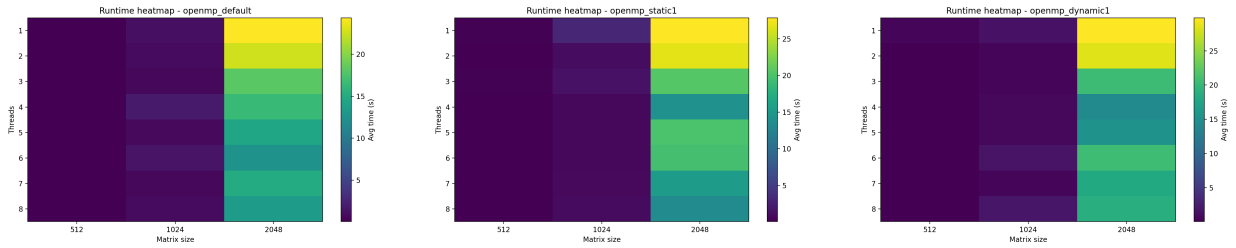
(a) default (b) static(1) (c) dynamic(1)

图 6: OpenMP 三种调度策略加速比曲线



(a) default (b) static(1) (c) dynamic(1)

图 7: OpenMP 三种调度策略并行效率曲线



(a) default (b) static(1) (c) dynamic(1)

图 8: OpenMP 三种调度策略热力图

6.2 OpenMP 结果分析

从 8 线程结果看，`openmp_default` 在三个规模下都给出了最短运行时间：当矩阵规模为 512 时，`default`、`static(1)` 和 `dynamic(1)` 分别约为 0.0592s、0.0664s 和 0.1059s；当规模提升到 1024 时，三者分别约为 0.7956s、0.8873s 和 1.8009s；当规模为 2048 时，三者分别约为 13.3494s、13.4750s 和 18.8711s。可以看到，`static(1)` 在大规模下已经接近 `default`，而 `dynamic(1)` 在本实验这种规则、均匀的任务上始终更慢。

若只看 2048 规模下各版本的最佳线程配置，`openmp_default` 在 6 线程时达到最优，为 12.5710s；`openmp_static1` 在 8 线程时达到 13.4750s；`openmp_dynamic1` 在 4 线程时达到 14.2550s。这个现象说明线程数不是单调收益源，线程继续增加后会受到调度、同步和内存带宽竞争影响，因此运行时间会出现波动。综合图和表来看，OpenMP 默认调度在本实验中最适合作为基线方案。

6.3 parallel_for 动态库验证

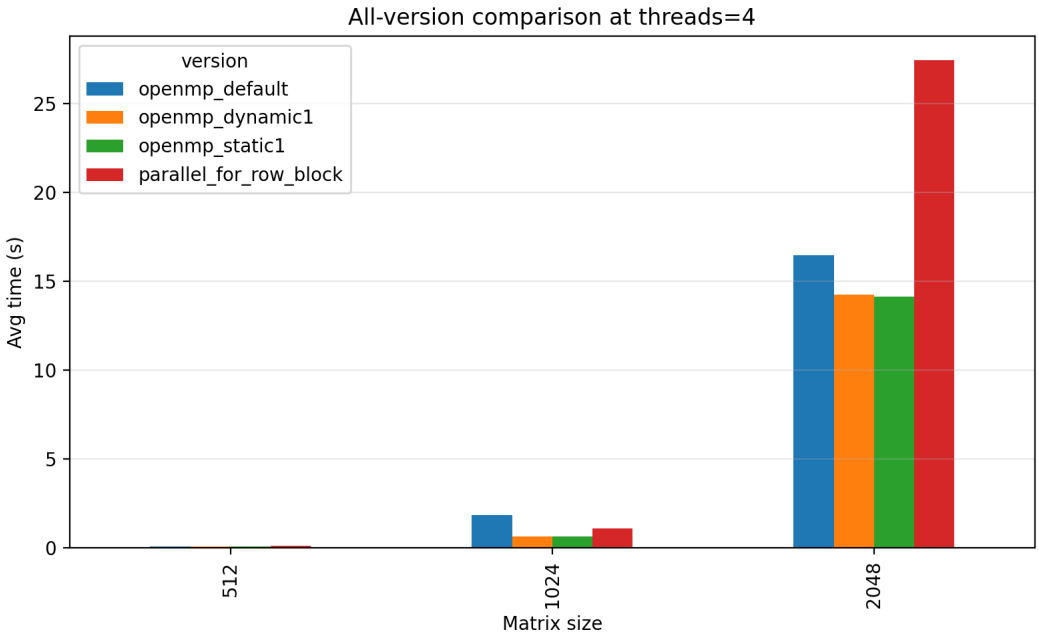
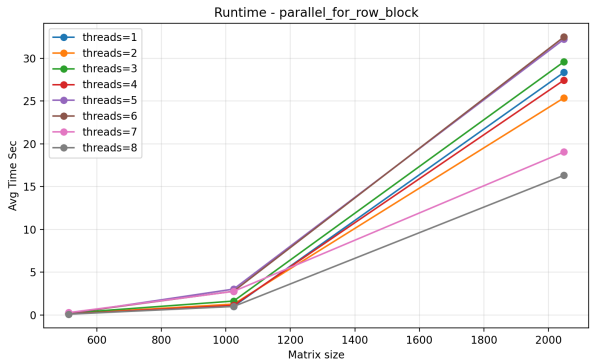
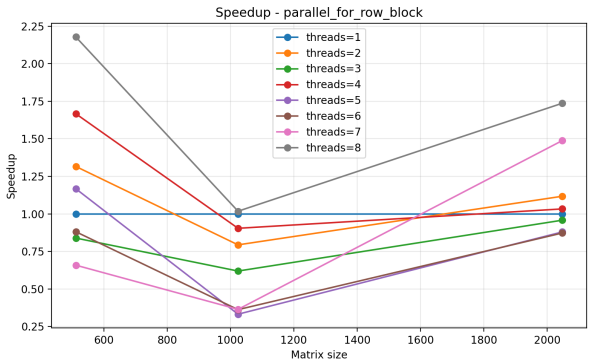


图 9: 全部版本在同一线程配置下的对比

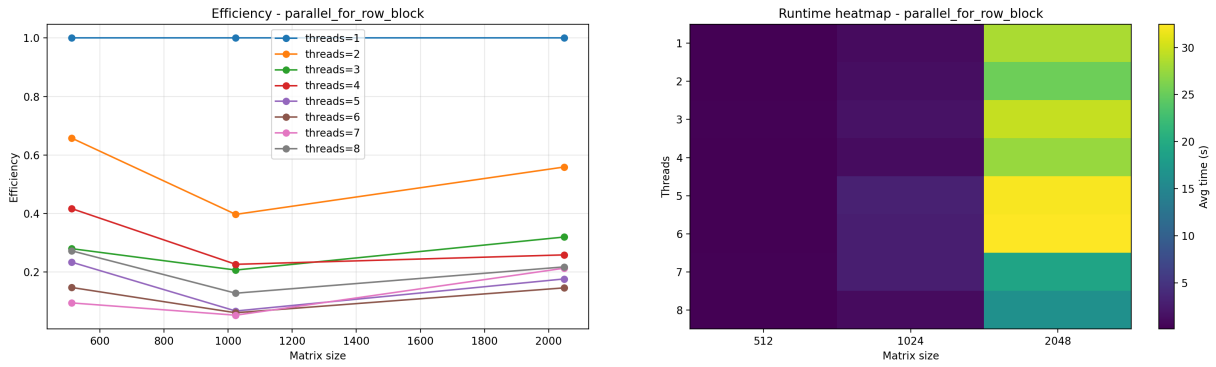


(a) 运行时间曲线



(b) 加速比曲线

图 10: parallel_for_row_block 性能结果



(a) 并行效率曲线 (b) 热力图

图 11: parallel_for_row_block 扩展特征

6.4 parallel_for 结果分析

parallel_for 动态库在 Docker Linux 环境中成功生成 libparallel_for.so，并被 parallel_for_matmul 正常加载执行。所有线程配置下的 checksum 与 OpenMP 三个版本保持一致，说明基于 Pthreads 的并行 for 分解在正确性上是成立的。

从性能上看，该版本在 2048 规模下的单线程时间约为 28.3465s，最佳结果出现在 8 线程，为 16.3184s，对应加速比约为 1.74。相比之下，2 线程仅提升到约 25.3660s，3 到 6 线程阶段还出现了波动甚至退化，直到 7 和 8 线程时才出现更明显的收益。这说明当前实现虽然完成了并行划分与动态库调用要求，但线程创建、销毁、同步以及连续块划分带来的负载不均和缓存竞争仍然比较明显，因此扩展性弱于 OpenMP。

将 parallel_for 与最佳 OpenMP 版本对比，在 2048 规模下，openmp_default 的最佳时间为 12.5710s，而该版本的最佳时间为 16.3184s，后者明显更慢。这一结果符合预期：OpenMP 运行时本身已经对线程调度和并行循环做了更成熟的优化，而本实验中的 parallel_for 仅实现了最基础的连续块划分机制，因此它更适合作为“并行 for 机制构造”的教学验证，而不是作为性能最优方案。

7 性能结论汇总

表 3: 最终性能结论汇总

比较对象	代表配置	结果	结论
OpenMP 三种调度	8 线程，512 / 1024 / 2048	default 均最快	默认调度在本实验中整体表现最好
default vs static(1)	2048，8 线程	13.3494s vs 13.4750s	static(1) 已接近 default，但仍略慢
default vs dynamic(1)	2048，8 线程	13.3494s vs 18.8711s	dynamic(1) 在规则任务上调度开销偏大
OpenMP 最佳配置	2048，全线程扫描	default 在 6 线程达到 12.5710s	线程数增加并非单调收益，存在最优点
`parallel_for` 扩展性	2048，1 线程到 8 线程	28.3465s 降到 16.3184s	能获得加速，但扩展效率明显弱于 OpenMP
OpenMP vs `parallel_for`	2048，各自最佳结果	12.5710s vs 16.3184s	手工构造并行 `for` 可行，但性能仍落后成熟运行时

8 结论

本实验完成了两条主线任务：一是用 OpenMP 实现并比较三种矩阵乘法调度策略，二是构造基于 Pthreads 的 `parallel_for` 动态库并用于矩阵乘法验证。实验结果表明，三种 OpenMP 版本和 `parallel_for` 版本都能给出一致的数值结果，说明实现正确。

在性能方面，OpenMP 默认调度整体最好。`schedule(static,1)` 在大规模下与默认调度接近，但总体仍略慢；`schedule(dynamic,1)` 由于调度开销较大，在本实验这种规则任务上没有体现优势。

对于 `parallel_for` 动态库，虽然它已经完成了 Linux `.so` 生成、外部程序调用和并行执行的要求，但其扩展效率弱于 OpenMP，说明手工构造的基础版并行 `for` 机制在功能上可行，在性能上仍有进一步优化空间。